

SQL: Structured Query Language

SQL Editor

```
SELECT employee_id, first_name, last_name,  
       department  
FROM employees  
WHERE status = "ACTIVE"  
ORDER BY last-name ASC;
```

Linguagem Declarativa

O usuário especifica o que deseja, não como obter.

DDL

Define estruturas e esquemas do banco de dados.

DML

Consulta, insere, atualiza e exclui dados.

DDL (Data Definition Language) é o subconjunto do SQL responsável por definir e gerenciar a estrutura do banco de dados.

Esquema das Relações

Define a estrutura das tabelas e seus relacionamentos.

Restrições de Integridade

Garante a consistência e validade dos dados armazenados.

Domínios dos Atributos

Especifica os tipos de dados permitidos para cada coluna.

Índices e Segurança

Otimiza consultas e controla o acesso aos dados.

Tipos Textuais

- char(n) - texto de tamanho fixo
- varchar(n) - texto de tamanho variável

Tipos Numéricos

- int - números inteiros
- float, double - números decimais

Tipos Temporais

- date - datas
- time - horários
- timestamp - data e hora combinados

agencia

nomeAgencia, cidadeAgencia, ativo

conta

numeroConta, nomeAgencia, saldo

depositante

nomeCliente, numeroConta

cliente

nomeCliente, ruaCliente, cidadeCliente

emprestimo

numeroEmprestimo, nomeAgencia, quantia

tomador

nomeCliente, numeroEmprestimo

Inserir

Adicionar novos dados

Consultar

Recuperar informações

Atualizar

Modificar dados existentes

Excluir

Remover dados

Quando não precisamos de todos os atributos, podemos especificar apenas aqueles que desejamos visualizar.

```
SELECT nomeCliente, cidadeCliente  
FROM cliente;
```

Equivalente em Álgebra Relacional: $\pi_{\text{nomeCliente}, \text{cidadeCliente}}(\text{cliente})$

Exemplo 1: Filtro Numérico

```
SELECT *  
FROM conta  
WHERE saldo > 1200;
```

Álgebra Relacional: $\sigma_{saldo > 1200}(conta)$

Exemplo 2: Filtro em Empréstimos

```
SELECT *  
FROM emprestimo  
WHERE quantia >= 1000;
```

Álgebra Relacional: $\sigma_{quantia \geq 1000}(emprestimo)$

BETWEEN - Intervalo de Valores

```
SELECT numeroConta, saldo  
FROM conta  
WHERE saldo BETWEEN 500 AND  
2000;
```

Equivalente em Álgebra Relacional:

$$\pi_{\text{numeroConta}, \text{saldo}}(\sigma_{\text{saldo} \geq 500 \wedge \text{saldo} \leq 2000}(\text{conta}))$$

IS NULL - Valores Nulos

```
SELECT nomeCliente  
FROM cliente  
WHERE ruaCliente IS NULL;
```

Equivalente em Álgebra Relacional:

$$\pi_{\text{nomeCliente}}(\sigma_{\text{ruaCliente IS NULL}}(\text{cliente}))$$

LIKE - Correspondência de Padrões

```
SELECT nomeCliente  
FROM cliente  
WHERE nomeCliente LIKE 'S%';
```

Equivalente em Álgebra Relacional:

$$\pi_{\text{nomeCliente}}(\sigma_{\text{nomeCliente LIKE 'S\%'}}(\text{cliente}))$$

ORDER BY organiza resultados em ordem crescente (ASC) ou decrescente (DESC).

```
SELECT numeroConta, saldo  
FROM conta  
ORDER BY saldo DESC;
```

Equivalente em Álgebra Relacional: $\pi_{numeroConta, saldo}(conta)$ (ordenado por saldo DESC)

```
SELECT nomeCliente, numeroConta
FROM depositante, conta
WHERE depositante.numeroConta =
    conta.numeroConta;
```

Equivalente em Álgebra Relacional:

$$\pi_{\text{nomeCliente}, \text{numeroConta}}(\sigma_{\text{depositante.numeroConta} = \text{conta.numeroConta}}(\text{depositante} \times \text{conta}))$$

Filtragem Adicional

```
SELECT nomeCliente, saldo
FROM depositante, conta
WHERE depositante.numeroConta =
    conta.numeroConta
AND saldo > 1000;
```

Forma otimizada:

$$\pi_{\text{nomeCliente}, \text{saldo}}(\sigma_{\text{saldo} > 1000}(\text{depositante} \bowtie_{\text{depositante.numeroConta} = \text{conta.numeroConta}} \text{conta}))$$

Junção Implícita

```
SELECT d.nomeCliente, c.saldo
FROM depositante d, conta c
WHERE d.numeroConta = c.numeroConta
      AND c.nomeAgencia = 'Perryridge';
```

$$\pi_{nomeCliente, saldo} \left(\sigma_{depositante.numeroConta=conta.numeroConta \wedge conta.nomeAgencia='Perryridge'} (depositante \times conta) \right)$$

Junção Explícita

```
SELECT d.nomeCliente, c.saldo
FROM depositante d
JOIN conta c ON d.numeroConta = c.numeroConta
WHERE c.nomeAgencia = 'Perryridge';
```

$$\pi_{nomeCliente, saldo} \left(\sigma_{nomeAgencia='Perryridge'} (depositante \bowtie_{depositante.numeroConta=conta.numeroConta} conta) \right)$$

❏ Equivalentes em resultado, mas não computacionalmente.

O operador UNION combina resultados de duas ou mais consultas, eliminando duplicatas automaticamente.

```
SELECT nomeCliente  
FROM depositante  
UNION  
SELECT nomeCliente  
FROM tomador;
```

Equivalente em Álgebra Relacional:

$$\pi_{\text{nomeCliente}}(\text{depositante}) \cup \pi_{\text{nomeCliente}}(\text{tomador})$$

Use Aliases

Simplifique referências a tabelas com nomes curtos e descritivos.

Prefira JOIN Explícito

Torna o código mais legível e facilita a manutenção.

Especifique Colunas

Evite SELECT * em produção para melhor performance.

Filtre Adequadamente

Use WHERE para reduzir o volume de dados processados.

Ordene com Propósito

ORDER BY tem custo computacional, use quando necessário.

SELECT

Define quais colunas serão retornadas

FROM

Especifica as tabelas fonte dos dados

JOIN...ON

Combina tabelas relacionadas (opcional)

WHERE

Filtra registros com base em condições

ORDER BY

Organiza os resultados finais

Linguagem Declarativa

Foco no "o quê" obter, abstraindo a complexidade do acesso aos dados.

Filtragem e Projeção

WHERE filtra linhas e SELECT escolhe colunas dos resultados.

DDL e DML

DDL define estruturas e DML manipula os dados armazenados.

Junções

Combinam dados de múltiplas tabelas através de relacionamentos.

SUM

```
SELECT SUM(saldo)
FROM conta;
```

Álgebra Relacional: $\Gamma_{SUM(saldo)}(conta)$

COUNT

```
SELECT COUNT(*)
FROM conta;
```

Álgebra Relacional: $\Gamma_{COUNT(*)}(conta)$

AVG

```
SELECT AVG(saldo)
FROM conta;
```

Álgebra Relacional: $\Gamma_{AVG(saldo)}(conta)$

O GROUP BY organiza registros em grupos e permite aplicar funções agregadas a cada grupo individualmente.

```
SELECT nomeAgencia, AVG(saldo)
FROM conta
GROUP BY nomeAgencia;
```

Álgebra Relacional: $\text{nomeAgencia} \Gamma_{AVG(saldo)}(conta)$

Diferença entre WHERE e HAVING

WHERE filtra registros **antes** do agrupamento, enquanto HAVING filtra grupos **após** a agregação.

Exemplo Prático

```
SELECT nomeAgencia, AVG(saldo)
FROM conta
GROUP BY nomeAgencia
HAVING AVG(saldo) > 1000;
```

Equivalente em Álgebra Relacional:

$$\pi_{nomeAgencia, AVG(saldo)} \left(\sigma_{AVG(saldo) > 1000} \left(\Gamma_{nomeAgencia}^{AVG(saldo)}(conta) \right) \right)$$

Retorna apenas agências com saldo médio superior a 1000.

Características

- Executa independentemente da consulta externa
- Executada apenas uma vez
- Mais eficiente em termos de desempenho
- Resultado é reutilizado para todas as linhas

Álgebra Relacional: $\pi_{numeroConta}(\sigma_{saldo > \Gamma_{AVG(saldo)}(conta)}(conta))$

Exemplo: Contas Acima da Média

```
SELECT numeroConta
FROM conta
WHERE saldo > (
    SELECT AVG(saldo)
    FROM conta
);
```

O operador EXISTS verifica se uma subconsulta retorna algum resultado, retornando TRUE se pelo menos uma linha for encontrada.

```
SELECT nomeCliente
FROM cliente c
WHERE EXISTS (
  SELECT *
  FROM tomador t
  WHERE t.nomeCliente = c.nomeCliente
);
```

Equivalente em Álgebra Relacional (forma otimizada): $\pi_{nomeCliente}(Cliente \bowtie_{nomeCliente} Tomador)$

Cientes Sem Empréstimos

Identifica clientes que não possuem empréstimos registrados no sistema.

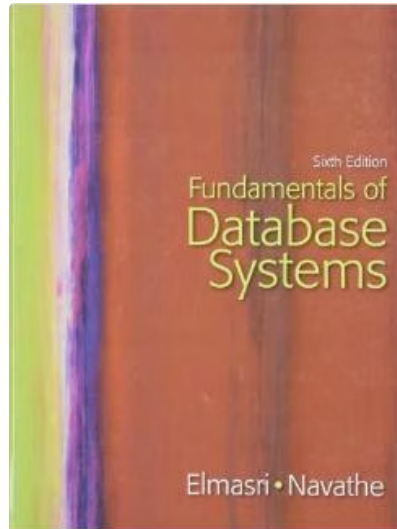
```
SELECT nomeCliente
FROM cliente c
WHERE NOT EXISTS (
  SELECT *
  FROM emprestimo e
  JOIN tomador t ON e.numeroEmprestimo = t.numeroEmprestimo
  WHERE t.nomeCliente = c.nomeCliente
)
```

Equivalente em Álgebra Relacional (forma otimizada):

$$\pi_{nomeCliente}(Cliente) - \pi_{nomeCliente}(Cliente \bowtie (Tomador \bowtie_{Tomador.numeroEmprestimo=Emprestimo.numeroEmprestimo} Emprestimo))$$

Análise de Risco

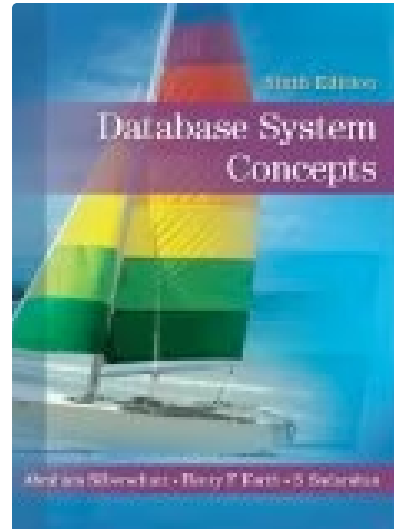
Útil para segmentar clientes por perfil de crédito e comportamento financeiro.



Elmasri & Navathe

Fundamentals of Database Systems

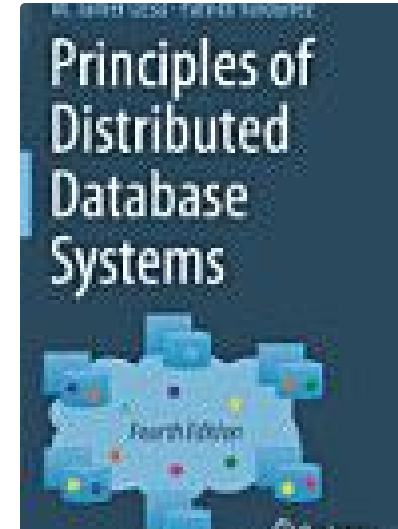
Pearson, 2016



Korth, Sudarshan & Silberschatz

Database System Concepts

McGraw-Hill, 2019



Özsü & Valduriez

Principles of Distributed Database Systems

Springer Nature, 2019